

Analysis and Recommendations for Dynamic Difficulty Adjustment in X Blocks

1. Introduction: The Crucial Role of Difficulty Balancing in Casual Puzzle Games

The Engagement Tightrope

The success of casual mobile games, particularly in the puzzle genre, hinges delicately on maintaining player engagement over extended periods. This requires navigating a fine line between challenge and accessibility, often conceptualized as the "flow channel".¹ Flow represents an optimal state of experience where the game's difficulty perfectly matches the player's skill level, leading to deep immersion and enjoyment. If the challenge is too low relative to the player's skill, boredom ensues; if the challenge significantly exceeds the player's capabilities, anxiety and frustration arise, often leading to player churn.² For games like *Block Blast!* and *X Blocks*, which rely on repeated sessions and long-term play, mastering this balance is not just desirable, but essential for retention and commercial viability. *Block Blast!*, for instance, explicitly aims to be a "classic companion" for its players, focusing on long-term relationships rather than short bursts of intense, potentially frustrating, gameplay.³

The User's Dilemma

The central issue prompting this analysis lies in the contrasting player experiences reported for *X Blocks* and *Block Blast!*. Players of *X Blocks* describe encountering frustrating end-game scenarios where the game grid becomes irrevocably filled ("grid lock"), leading to abrupt and unsatisfying session conclusions.⁴ Conversely, *Block Blast!* is perceived as managing its difficulty curve more gracefully; even when a game ends, players often feel encouraged to start another round, suggesting a less frustrating failure state [User Query]. This report aims to deconstruct the mechanisms potentially contributing to *Block Blast!*'s smoother perceived difficulty curve and provide actionable strategies for the *X Blocks* team to refine their own difficulty balancing, specifically addressing the grid-lock problem and enhancing player retention.

Defining DDA

A key technique for maintaining player flow is Dynamic Difficulty Adjustment (DDA), also known as Dynamic Game Difficulty Balancing (DGDB).¹ DDA involves the automatic, real-time adaptation of various game parameters, scenarios, or behaviors

based on an assessment of the player's current ability or state.¹ The fundamental goal of DDA is to sustain player interest and engagement throughout their interaction with the game by providing a consistently appropriate level of challenge.¹

2. Deconstructing Block Blast!'s Engagement Model: Beyond Simple Blocks

Core Loop & Philosophy

Block Blast! operates on a seemingly simple premise. Players are presented with an 8x8 grid and sets of three Tetris-like pieces, which they must drag and drop onto the grid.⁷ Unlike traditional Tetris, pieces do not fall automatically and cannot be rotated.⁸ The objective is to complete full horizontal or vertical lines, which then clear from the board, awarding points and freeing up space.⁸ The game offers a 'Classic Mode' for endless play and an 'Adventure Mode' featuring level-based objectives.³

Underpinning this core loop is Hungry Studio's distinct development philosophy. Their stated vision is "Bringing exceptional gaming experiences to a broader and more diverse audience," coupled with a focus on "experience equality".³ This translates into a design that intentionally avoids mechanics creating "excessive urgency or high-pressure time constraints".³ This philosophy strongly suggests that the game's DDA is deliberately tuned towards comfort, accessibility, and long-term play, rather than maximizing short-term difficulty or competitive pressure. This aligns with their aspiration to be a "classic companion" that integrates into players' daily relaxation routines.³

The "Block Algorithm" and AI

The apparent simplicity of *Block Blast!* belies a complex system working behind the scenes. Hungry Studio highlights their proprietary "Block Algorithm," described as incorporating "nearly 100 nested sub-rules" that function simultaneously to generate "dynamic and personalized gameplay experiences".³ This indicates a highly sophisticated approach to managing the game state and piece generation.

Further enhancing this system is the "Block AI Robot." This AI employs deep network modeling technology, analyzing vast numbers of game states and player actions to learn behavioral patterns and predict likely player responses.³ Notably, this AI achieves an "80% alignment with real player behaviors".³ The combination of numerous intricate rules with an AI capable of predictive modeling suggests a DDA system far more nuanced than simple reactive triggers. It likely evaluates a multitude of factors simultaneously – such as the current board complexity (e.g., grid density,

fragmentation, number of potential line clears), recent player performance (e.g., clearing speed, score changes), and possibly historical piece placement patterns – to subtly influence the *probability* of which pieces appear next. The 80% alignment figure is particularly revealing; it implies the system doesn't merely mimic the player or force optimal moves but allows for some deviation, preserving a degree of perceived randomness and player agency while gently guiding the experience away from frustrating dead-ends. This sophisticated, data-driven approach directly supports the goal of creating a personalized, long-term, and less frustrating experience.

Observed DDA Manifestations

Player observations and game analyses reveal tangible effects likely stemming from this underlying DDA system:

- **Helpful Pieces:** Multiple sources note that the game provides blocks that "surprisingly often match your missing space precisely".⁷ This suggests the DDA proactively intervenes during standard gameplay, subtly biasing piece generation towards blocks that are placeable and potentially beneficial, contributing to the feeling of power and smooth progress.
- **"God Mode":** A more overt intervention, described as "God Mode," reportedly activates after a player "loses multiple times".¹¹ In this state, the game "drops ideal pieces," effectively ensuring the player can continue progressing, albeit at the cost of diminished challenge.¹¹ This functions as a crucial safety net, preventing players from hitting insurmountable walls of frustration that could lead to churn.

The presence of both subtle, continuous assistance ("helpful pieces") and a stronger, triggered intervention ("God Mode") points towards a *tiered DDA strategy* in *Block Blast!*.

- **Tier 1:** Operates constantly, driven by the complex Block Algorithm and AI. It analyzes the immediate game state and recent player performance to make fine-grained adjustments to piece probabilities, aiming to keep the player in the flow channel during normal play.
- **Tier 2:** Acts as a safety net, triggered only when the system detects a high degree of player struggle (e.g., multiple consecutive low-scoring losses, prolonged periods without clears, or perhaps specific board states indicating imminent failure). This tier provides more direct assistance ("ideal pieces") to break the failure cycle and prevent the player from abandoning the game.

This tiered approach effectively balances the need to maintain engagement and a sense of accomplishment during typical gameplay with the imperative to mitigate extreme frustration and retain players, particularly those in the casual target

audience.

Player Experience & Contradictions

Block Blast! boasts impressive engagement metrics, with reported figures exceeding 40 million Daily Active Users (DAU) and 150 million Monthly Active Users (MAU) as of late 2024.³ This success is achieved despite a heavy reliance on ad-based monetization, including interstitial and banner ads, as well as rewarded videos for revives.⁷ The developers note an interesting pattern of players returning seasonally, suggesting strong long-term appeal.³

User feedback reflects this duality. Many reviews praise the game as fun, relaxing, addictive, and a good way to pass the time.⁸ Some users specifically noted improvements after game updates, particularly regarding the point system and block assigning algorithm.⁸ However, a significant volume of negative feedback exists, primarily centered on the intrusive and frequent advertising.¹³ Other complaints include receiving seemingly impossible blocks that force a loss¹⁴, the game feeling overly reliant on luck rather than strategy¹³, and technical issues like black screens after ads or, specifically for Android users, starting games with partially filled boards.⁸

The high engagement levels, combined with the positive feedback on the core loop's relaxing nature, suggest that the DDA is largely successful in its primary goal: making the *core puzzle gameplay* feel non-frustrating and engaging for its intended broad, casual audience. This contributes significantly to retention, potentially offsetting frustrations caused by the aggressive monetization strategy. The negative feedback often points to factors *external* to the core DDA (ads, potential bugs like the Android starting board issue⁸) or stems from a mismatch in player expectations. Players seeking a deep, consistently high-skill challenge may find the DDA's interventions (like God Mode or frequent helpful pieces) overly simplistic or even patronizing.¹¹ The perception of "impossible blocks"¹⁴ could arise from genuine randomness, occasional DDA miscalculations, or situations where the algorithm prioritizes preventing immediate grid lock (by providing *any* placeable piece) over providing the *optimal* piece for scoring or board setup. This prioritization aligns with the long-term, low-frustration design philosophy, even if it doesn't satisfy players focused purely on maximizing score or strategic depth.

3. Diagnosing X Blocks' End-Game Frustration: When the Grid Locks Up

Core Loop & Reported Issues

Based on available descriptions and user reviews, *X Blocks* offers a variety of block puzzle modes, including mechanics like merging, sliding, dropping, and stacking blocks.⁴ The overarching goal appears to be achieving high scores across these different modes.⁴

However, the primary user concern, and the impetus for this analysis, is the frustrating nature of the end-game. Players report that the grid frequently fills up and remains in a locked state, creating an unavoidable and unsatisfying loss condition [User Query]. This contrasts sharply with the perceived smoother experience in *Block Blast!*. User reviews corroborate this issue, with players expressing frustration after game updates changed the mechanics.⁴ Specific complaints include pieces feeling "not random" and exhibiting a "huge imbalance" ⁴, shapes being provided in sequences that make it impossible to keep spaces open ⁴, and, most critically, a potential issue where completed blocks do not disappear from the grid.⁴ If the latter is accurate, it represents a fundamental mechanical flaw drastically increasing difficulty and frustration.

Analysis of Failure State

The core difference appears to lie in how each game manages impending failure. While *Block Blast!* seems to utilize its DDA to proactively steer players away from unavoidable failure states (via helpful pieces) and provides a safety net for when failure does occur (God Mode), *X Blocks* seemingly lacks effective mechanisms to prevent or mitigate grid lock. The result is likely a higher frequency of abrupt, unavoidable losses where the player feels powerless.

Furthermore, the comments regarding "non-random" and "imbalanced" pieces in *X Blocks* ⁴ are particularly concerning. Players perceive the piece generation algorithm as working *against* them, presenting unfair or deliberately obstructive sequences. This is the antithesis of a well-implemented DDA, which should feel supportive or, at worst, neutrally random. *Block Blast!*'s DDA, despite its interventions, appears to largely succeed in feeling helpful or at least not actively punitive.

The central problem in *X Blocks*, therefore, seems to be a lack of effective DDA, particularly in managing the board state as it approaches the end-game. Without systems to analyze the board, detect rising player struggle, and subtly adjust piece generation to favor placeable or helpful blocks, the game inevitably falls into unwinnable scenarios more often and with less warning than *Block Blast!*. This creates feelings of helplessness and unfairness, directly contributing to player frustration and likely impacting retention negatively. The perception of the game's systems actively hindering the player, rather than supporting them, represents a critical design failure

compared to *Block Blast!*'s player-centric approach.

4. Mastering the Flow: Principles of Effective Dynamic Difficulty Adjustment

The Goal: The Flow Channel

As established, the primary objective of DDA is to maintain player engagement by keeping them within the "flow channel".¹ This psychological state occurs when the perceived challenges of the game align appropriately with the player's perceived skill level. DDA systems aim to dynamically adjust the game's challenge to prevent the player from straying into boredom (where skill significantly outweighs challenge) or anxiety/frustration (where challenge significantly outweighs skill).² A crucial aspect of successful DDA is subtlety; ideally, the adjustments should occur without the player consciously noticing them.² Overt or poorly implemented DDA can feel manipulative, condescending, or break immersion, undermining its intended purpose.¹⁷

Key DDA Approaches for Puzzle Games

Several methods can be employed to dynamically adjust difficulty in games ¹:

- **Parameter Tuning:** Modifying numerical values within the game. In classic *Tetris*, this is primarily the speed at which blocks fall.²⁰ Other examples could include adjusting scoring rules, the time limit, or the health/strength of elements (less common in single-player puzzle games).
- **Behavior Modification:** Altering the behavior of non-player characters (NPCs) or AI opponents. This is highly relevant in genres like shooters or strategy games but less applicable to typical single-player block puzzlers.
- **Environment/Content Generation:** Adapting the game world or the content presented to the player. This can involve changing level layouts, enemy placements, or, most pertinently for block puzzle games, **dynamically adjusting the sequence or probability of the pieces generated**.¹ Research into *Tetris* variants has explored algorithms that select pieces based on the current board state to either help (NICETRIS) or hinder (BUST HEAD) the player.²⁰ This adaptive piece selection is the most promising DDA avenue for addressing *X Blocks*' grid-lock issue.
- **Assistance Techniques:** Providing direct help to the player. This can range from subtle hints and visual cues ²¹ to more direct aids like aim assistance in shooters, temporary power-ups ¹, or features like an "Undo" button.¹¹

Metrics for Player State Tracking in Block Puzzles

Effective DDA relies on accurate assessment of the player's current state and skill level. Simply tracking wins and losses is insufficient. For block puzzle games, more nuanced metrics are required to understand player performance and predict potential frustration points like grid lock:

- **Performance Metrics:** These quantify how well the player is doing. Examples include:
 - Clear Speed: Lines cleared per unit of time.¹
 - Combo Frequency/Score Velocity: How often the player clears multiple lines at once and the rate of score increase.⁶
- **Struggle Metrics:** These indicate when the player is having difficulty. Examples include:
 - Time Since Last Clear: A long duration suggests the player is struggling to make progress.²⁰
 - Moves Without Clear: Similar to time, tracking unproductive moves.
 - Failure Rate: Frequency of game overs, restarts, or use of revives.¹
 - Unplaceable Pieces Received: How often the player is given pieces that cannot fit anywhere on the current board.
- **Board State Metrics:** These analyze the complexity and potential difficulty of the current grid configuration. Examples include:
 - Grid Density: The percentage of the board currently occupied by blocks.²⁰ High density often precedes grid lock.
 - Available Placements: The number of valid locations where the *current* or *upcoming* set of pieces can be placed.²⁵ A low number, especially zero for any given piece, signals imminent danger.
 - Isolated Holes: Empty cells surrounded by filled cells, which are notoriously difficult to fill.²³
 - Board "Bumpiness": Adapted from *Tetris* research, this measures the height variance between adjacent columns. A high bumpiness value indicates an uneven, difficult-to-manage board state.²⁰

Measuring the right parameters is fundamental for effective DDA. To combat the specific problem of grid lock in *X Blocks*, metrics assessing the immediate risk – such as board density, the number of available placements for upcoming pieces, and the time since the last line clear – are likely more critical than overall score or historical win rate. Tracking these allows a DDA system to anticipate potential grid-lock scenarios and intervene proactively *before* the player reaches an unwinnable state.

Piece Generation Strategies

The method used to determine which pieces appear next significantly impacts

difficulty and perceived fairness:

- **True Random:** Each piece has an equal and independent chance of appearing. While fair in theory, this can lead to frustratingly long "droughts" of needed pieces or "floods" of unhelpful ones.²⁰ User reviews suggest *Block Blast!* might feel like this at times, despite its underlying algorithm.¹³
- **Bag Randomizer:** A common approach in modern *Tetris* games.²⁷ Pieces are drawn without replacement from a "bag" containing one of each type (e.g., a 7-bag). Once the bag is empty, it's refilled. This guarantees that players won't go too long without seeing a specific piece type, reducing frustration from droughts, but it also makes the sequence somewhat predictable.²⁷
- **Adaptive/Algorithmic Selection:** As employed by *Block Blast!*³ and explored in *Tetris* research²⁰, this method dynamically adjusts the probability of different pieces appearing based on the current game state and/or player performance metrics. This allows the system to subtly help players out of difficult situations (like NICETRIS²⁰ or *Block Blast!*'s helpful pieces⁷) or increase the challenge.

To address the grid-lock frustration and perceived unfairness in *X Blocks*, transitioning from a potentially simple random or bag system towards an *adaptive* piece generation algorithm is strongly recommended. This algorithm should leverage the board state and player struggle metrics discussed previously. When the system detects that the player is nearing a grid-lock state (e.g., high density, few available placements, long time without a clear), it should subtly increase the probability of generating pieces that are not only placeable but also potentially helpful – pieces that fit existing gaps, clear lines, or reduce board fragmentation and bumpiness. This proactive, state-aware approach offers the best chance of replicating the smoother difficulty curve perceived in *Block Blast!*.

5. Architecting a Player-Centric DDA for X Blocks

Goal

The objective is to design a DDA system for *X Blocks* that effectively prevents frustrating grid-lock scenarios, maintains player engagement through balanced challenge, and feels fair and supportive. This design draws inspiration from the inferred mechanisms in *Block Blast!* and established DDA principles, tailored to address the specific issues reported by *X Blocks* players.

Core Strategy: Adaptive Piece Generation

The primary focus of the proposed DDA system is the dynamic adjustment of the probability distribution for the next set of pieces offered to the player. This approach

directly targets the core issue of receiving unhelpful or unplaceable blocks leading to grid lock. The system should continuously analyze the Board State and Player State (using relevant metrics) to compute a real-time Difficulty/Struggle Score. This score will then modulate the likelihood of different piece types or categories appearing in the upcoming set.

Key Metrics to Track

Building upon the principles outlined in Section 4, the following metrics are crucial for informing the DDA decisions in *X Blocks*:

1. **Board_Density:** Percentage of grid cells filled (e.g., 0.0 to 1.0). Higher values increase the Struggle_Score.
2. **Available_Placements:** The total number of unique, valid positions on the current board where the *next* set of pieces (assuming a lookahead of 3 pieces, similar to *Block Blast!*⁷) could be placed. Critically, this should also track if *any* piece in the upcoming set has zero valid placements. Fewer placements drastically increase the Struggle_Score.
3. **Time_Since_Clear:** Measured in seconds or number of moves since the last row/column clear. Longer times increase the Struggle_Score.
4. **Isolated_Holes:** Count of empty cells completely surrounded by filled cells. More holes increase the Struggle_Score.
5. **Board_Bumpiness:** Sum of absolute height differences between adjacent columns (adapting the *Tetris* metric²⁰). Higher bumpiness increases the Struggle_Score.
6. **Recent_Failure_Rate:** Frequency of game overs or potentially player-initiated restarts within the current session or recent time window. A high rate could trigger Tier 2 intervention.

DDA Logic - Tier 1 (Subtle Adjustment)

This tier operates continuously to gently guide the player experience:

- **Struggle Score Calculation:** Compute a Struggle_Score (e.g., ranging from 0 to 1) based on a weighted combination of the normalized metrics above. For example, Available_Placements might receive a higher negative weight as it approaches zero, while Board_Density and Time_Since_Clear contribute positively. Smoothing techniques (like a moving average) should be applied to prevent jarring, rapid shifts in difficulty.¹⁹
- **Piece Categorization:** Define categories for the available block shapes based on their general utility or difficulty to place (e.g., 'Easy': 1x1, 1x2, 1x3, 2x2; 'Medium': L-shapes, T-shapes; 'Hard': S/Z shapes, larger complex shapes²⁵). Assign

baseline probabilities to each category reflecting their natural frequency or desired balance.

- **Probability Adjustment:**

- If Struggle_Score is low (player is doing well): Use baseline probabilities or slightly favor 'Medium'/'Hard' pieces to maintain challenge.
- If Struggle_Score is moderate/rising: Subtly increase the probability of drawing pieces from the 'Easy' and 'Medium' categories. Crucially, the system should prioritize pieces within these categories that *have confirmed valid placements* on the current board. Simultaneously, slightly decrease the probability of 'Hard' pieces, especially those identified as having few or zero valid placements in the current board state.
- The adjustment should be gradual and proportional to the Struggle_Score. The goal is to slightly bias the odds in the player's favor when they are struggling, making it *more likely* they receive a useful piece, without guaranteeing it or making the intervention obvious.

- **Board-Specific Heuristics (Advanced):** Consider incorporating analysis similar to *Tetris*'s contour analysis.²⁰ When the board is messy (high bumpiness, many holes), the DDA could slightly favor pieces known to fill gaps effectively or smooth the terrain (e.g., straight pieces for reducing bumpiness).

DDA Logic - Tier 2 (Safety Net - Optional but Recommended)

This tier activates only in critical situations to prevent almost certain failure and player churn:

- **Trigger Conditions:** Activate if the Struggle_Score surpasses a high critical threshold (e.g., > 0.85) AND other conditions are met (e.g., Board_Density $> 90\%$, Available_Placements for an upcoming piece is zero), OR if the Recent_Failure_Rate spikes significantly (e.g., 3+ losses in quick succession).
- **Intervention:** When triggered, the system provides more direct assistance. Options include:
 - **"Ideal Pieces" (Block Blast! approach ¹¹):** Guarantee the next set contains one or two highly useful pieces (e.g., a 1x1 block, a short line piece) that are guaranteed to be placeable and likely help clear a line or fill a critical gap.
 - **"Pity Piece":** After a set number of consecutive moves without any line clear, or after receiving multiple consecutive sets containing unplaceable pieces, guarantee the next single piece generated is simple and placeable (e.g., a 1x1).
- **Implementation Considerations:** This tier should be used sparingly. Its activation should ideally be invisible or very subtly signaled to the player, as overt

"saving" can diminish the sense of accomplishment.¹¹ The primary function is churn prevention in moments of extreme, likely system-induced, frustration.

Proposed Table: DDA Parameter Comparison (Block Blast! vs. Proposed X Blocks)

Parameter	Block Blast! (Inferred)	X Blocks (Proposed)	Rationale for X Blocks Proposal
Primary DDA Goal	Long-term engagement, low frustration, "classic companion" ³	Prevent grid lock, reduce end-game frustration, maintain flow, improve retention	Directly addresses the core user complaint and aligns with best practices for casual puzzle game retention.
Core DDA Mechanism	Adaptive Piece Generation via complex "Block Algorithm" & AI ³	Adaptive Piece Generation via weighted metrics & probability adjustment	Focuses the DDA on the most impactful factor for grid lock: the pieces provided to the player.
Key Metrics	Likely includes board state, player performance, AI predictions ³	Board_Density, Available_Placements, Time_Since_Clear, Isolated_Holes, Board_Bumpiness, Recent_Failure_Rate	Provides concrete, measurable inputs specifically chosen to anticipate and mitigate grid-lock scenarios.
Tier 1 Adjustment	Subtle, continuous probability shifts ("helpful pieces") ⁷	Subtle, continuous probability shifts based on calculated Struggle_Score	Mimics the successful subtle approach, aiming to guide players away from failure without being obvious.
Tier 1 Bias	Towards placeable/helpful pieces when struggling	Towards placeable/helpful pieces (Easy/Medium categories) when Struggle_Score rises	Explicitly defines the desired bias to counteract the current perception of unfair piece generation.

Tier 2 Trigger	"Multiple losses" ¹¹	Critical Struggle_Score threshold + board state (density/placements) OR high Recent_Failure_Rate	Provides more specific, data-driven triggers based on imminent failure risk or demonstrated repeated struggle.
Tier 2 Intervention	"Drops ideal pieces" ¹¹	Guarantee next set contains highly useful/placeable pieces OR "Pity Piece" mechanism	Offers concrete safety net options to break failure loops, directly preventing the frustrating grid-lock end-state.
Perceived Fairness	Generally positive, though some find it too easy/luck-based ¹¹	Aims for high perceived fairness by being subtly supportive, avoiding obstructive pieces	Addresses the critical negative feedback about unfairness in X Blocks by designing the DDA to assist, not hinder.
Tuning Method	Extensive A/B testing (50+/week) ³	Requires extensive A/B testing, metric analysis, and playtesting	Emphasizes the iterative process needed to balance the DDA effectively, mirroring successful industry practices.

This comparative framework translates the analysis into a concrete specification for *X Blocks*. It highlights the necessary shift from potentially passive or punitive difficulty management to a proactive, player-supportive DDA strategy focused on preventing the specific frustration points identified by users.

6. Implementation Pathways: Bringing Adaptive Difficulty to Life

Implementing the proposed DDA system requires careful consideration of data tracking, algorithmic calculation, and robust testing.

Data Tracking Infrastructure

The foundation of the DDA system is the ability to accurately track the key metrics identified in Section 5 during gameplay. This necessitates:

- **Board State Analysis:** After each piece placement, the system must efficiently

analyze the 8x8 grid to calculate Board_Density, identify Isolated_Holes, and compute Board_Bumpiness.

- **Placement Lookahead:** Before generating the next set of pieces, the system needs to determine Available_Placements for potential upcoming pieces. This requires iterating through possible pieces and their orientations to check for valid placements on the current board state. This lookahead is crucial for predicting imminent grid lock.
- **Player State Tracking:** The system must maintain session-level data for each player, including Time_Since_Clear (updated after each move and reset on clear) and Recent_Failure_Rate (tracking game overs or restarts).

Calculating the Struggle Score

The Struggle_Score serves as the primary input for the Tier 1 DDA adjustments. Its calculation involves several steps:

1. **Metric Normalization:** Convert each raw metric (e.g., density from 0-1, placements from 0 to max possible, time in seconds) to a common scale, typically 0 to 1, where 1 represents the highest level of struggle for that metric (e.g., high density, low placements, long time).
2. **Weighting:** Assign weights to each normalized metric based on its perceived importance in contributing to player struggle, especially concerning grid lock. For instance, Available_Placements approaching zero might warrant a significantly higher weight than Board_Bumpiness. These weights are critical tuning parameters.
3. **Combination:** Combine the weighted, normalized metrics into a single Struggle_Score. A simple linear combination is a starting point:
$$\text{Struggle_Score} = (w1 * \text{norm}(\text{Density}) + w2 * (1 - \text{norm}(\text{Available_Placements})) + w3 * \text{norm}(\text{Time_Since_Clear}) + w4 * \text{norm}(\text{Isolated_Holes}) + w5 * \text{norm}(\text{Board_Bumpiness}))$$
4. **Smoothing:** Apply a smoothing function, such as a moving average over the last N calculated scores, to the Struggle_Score. This prevents overly sensitive or rapid fluctuations in difficulty adjustments based on single moves, leading to a more stable player experience.¹⁹

Pseudocode Example: Adaptive Piece Probability Adjustment (Tier 1)

Code snippet

```

// --- Constants ---
const BASELINE_PROBS = { Easy: 0.4, Medium: 0.4, Hard: 0.2 } // Example baseline
distribution
const MAX_ADJUSTMENT_FACTOR = 0.1 // Max probability shift towards Easy
const STRUGGLE_THRESHOLD_LOW = 0.3 // Below this, no adjustment
const STRUGGLE_THRESHOLD_HIGH = 0.7 // Above this, max adjustment

// --- Main Function ---
// Called before generating the next set of 3 pieces
function generate_next_pieces(current_board_state, player_state):

    // 1. Calculate current smoothed struggle score (0 to 1)
    struggle_score = calculate_smoothed_struggle_score(current_board_state,
player_state)

    // 2. Determine adjustment factor based on struggle score
    adjustment_factor = 0.0
    if struggle_score > STRUGGLE_THRESHOLD_LOW:
        // Linearly scale adjustment between low and high thresholds
        scale = min(1.0, (struggle_score - STRUGGLE_THRESHOLD_LOW) /
(STRUGGLE_THRESHOLD_HIGH - STRUGGLE_THRESHOLD_LOW))
        adjustment_factor = scale * MAX_ADJUSTMENT_FACTOR

    // 3. Calculate adjusted probabilities
    adjusted_probs = {}
    adjusted_probs[Easy] = BASELINE_PROBS[Easy] + adjustment_factor
    adjusted_probs[Medium] = BASELINE_PROBS[Medium] // Keep medium stable for
simplicity
    adjusted_probs[Hard] = BASELINE_PROBS[Hard] - adjustment_factor

    // Ensure probabilities are valid (non-negative and sum to 1)
    adjusted_probs = normalize_probabilities(adjusted_probs)

    // 4. Generate pieces based on adjusted probabilities, prioritizing placeable ones
    next_pieces =
    potential_pieces_pool = get_all_piece_types() // Get list of all possible piece shapes

    for i from 1 to 3:
        // Select category probabilistically

```



```

selected_category = select_category_based_on_probability(adjusted_probs)

// Get pieces of that category
category_pieces = filter_pieces_by_category(potential_pieces_pool,
selected_category)

// --- Crucial Step: Find a PLACEABLE piece ---
found_piece = null
shuffled_category_pieces = shuffle(category_pieces) // Randomize order within
category

for piece in shuffled_category_pieces:
    if can_place(piece, current_board_state): // Check if *any* valid placement exists
        found_piece = piece
        break // Found a placeable piece in the selected category

// If no placeable piece found in the target category, try preferred fallback
categories
if found_piece is null:
    fallback_categories = [Easy, Medium] // Prioritize easier pieces if struggling
    for fallback_category in fallback_categories:
        fallback_category_pieces = filter_pieces_by_category(potential_pieces_pool,
fallback_category)
        shuffled_fallback_pieces = shuffle(fallback_category_pieces)
        for piece in shuffled_fallback_pieces:
            if can_place(piece, current_board_state):
                found_piece = piece
                break
    if found_piece is not null:
        break

// If still no placeable piece found (rare, indicates near-failure state)
if found_piece is null:
    // Find *any* placeable piece, prioritizing simplest (e.g., 1x1)
    found_piece = find_simplest_placeable_piece(current_board_state,
potential_pieces_pool)
    // If even this fails, the board is truly locked for *any* piece. Game over.
    if found_piece is null:
        // Handle game over condition upstream

```

```
    return generate_unplaceable_set() // Or signal failure
```

```
    next_pieces.append(found_piece)
```

```
    return next_pieces
```

```
// --- Helper Functions (Conceptual) ---
```

```
function calculate_smoothed_struggle_score(board, player):
```

```
    // Calculate raw score based on weighted, normalized metrics (density, placements,  
    time_stuck, etc.)
```

```
    raw_score = calculate_raw_struggle_score(board, player)
```

```
    // Apply moving average or exponential smoothing using player.historical_scores
```

```
    smoothed_score = apply_smoothing(raw_score, player.historical_scores)
```

```
    return smoothed_score
```

```
function normalize_probabilities(probs):
```

```
    // Ensure all values >= 0 and sum == 1.0
```

```
    //... implementation...
```

```
    return normalized_probs
```

```
function select_category_based_on_probability(probs):
```

```
    // Use weighted random selection based on category probabilities
```

```
    //... implementation...
```

```
    return selected_category // e.g., Easy, Medium, or Hard
```

```
function filter_pieces_by_category(all_pieces, category):
```

```
    // Return list of piece shapes belonging to the specified category
```

```
    //... implementation...
```

```
    return category_pieces
```

```
function can_place(piece, board):
```

```
    // Iterate through all possible (x, y) positions and orientations (if applicable)
```

```
    // Return true if at least one valid placement exists without overlap
```

```
    //... implementation...
```

```
    return true or false
```

```
function find_simplest_placeable_piece(board, all_pieces):
```

```
    // Iterate through pieces, ordered by simplicity (e.g., smallest area first)
```

```
    // Return the first one found that is placeable
```

```
//... implementation...  
return piece or null
```

Pseudocode Example: Simple "Near-Fail State" Intervention (Tier 2 Trigger)

Code snippet

```
// --- Constants ---  
const CRITICAL_STRUGGLE_SCORE = 0.85  
const CRITICAL_DENSITY = 0.90  
const MIN_AVAILABLE_PLACEMENTS = 1 // Threshold for *any* piece in next potential  
set  
const CONSECUTIVE_LOSS_THRESHOLD = 3  
  
// --- Check Function ---  
// Called after each move or before generating pieces  
function check_and_trigger_safety_net(current_board_state, player_state):  
  
    // Calculate necessary metrics  
    struggle_score = calculate_smoothed_struggle_score(current_board_state,  
player_state)  
    density = calculate_density(current_board_state)  
    // Look ahead to see if *any* piece in the next hypothetical set would be unplaceable  
    min_placements_next_set =  
check_min_placements_for_next_set(current_board_state)  
  
    trigger = false  
    reason = ""  
  
    // Condition 1: High struggle and density  
    if struggle_score > CRITICAL_STRUGGLE_SCORE and density > CRITICAL_DENSITY:  
        trigger = true  
        reason = "High Struggle and Density"  
  
    // Condition 2: Imminent grid lock based on lookahead  
    if not trigger and min_placements_next_set < MIN_AVAILABLE_PLACEMENTS:  
        trigger = true
```

```

    reason = "Imminent Grid Lock"

// Condition 3: Repeated failures
if not trigger and player_state.consecutive_losses >=
CONSECUTIVE_LOSS_THRESHOLD:
    trigger = true
    reason = "Multiple Losses"

// If any condition met, activate the safety net
if trigger:
    activate_safety_net_flag(player_state) // Set a flag for next piece generation
    log_event("SafetyNetTriggered", {"Reason": reason, "StruggleScore":
struggle_score, "Density": density})
    // Reset consecutive losses if triggered by that
    if reason == "Multiple Losses":
        player_state.consecutive_losses = 0
    return true

return false

// --- Modification to Piece Generation ---
// Modify generate_next_pieces() to check the flag set by activate_safety_net_flag()
function generate_next_pieces(current_board_state, player_state):
    if player_state.is_safety_net_active:
        // Generate a guaranteed helpful set (e.g., 1x1, 1x2 line pieces known to be
placeable)
        next_pieces = generate_guaranteed_helpful_set(current_board_state)
        deactivate_safety_net_flag(player_state) // Reset flag after use
        return next_pieces
    else:
        // Proceed with normal Tier 1 adaptive generation
        //... (as described in previous pseudocode)...

// --- Helper Functions (Conceptual) ---
function check_min_placements_for_next_set(board):
    // Simulate generating potential next pieces (using baseline or current probs)
    // For each potential piece, check min number of placements available
    // Return the minimum count found across the potential set
    //... implementation...

```

```
return min_placement_count
```

```
function generate_guaranteed_helpful_set(board):
```

```
// Generate a set of 3 simple pieces (e.g., 1x1, 1x2) guaranteed to be placeable
```

```
//... implementation...
```

```
return helpful_pieces
```

Implementation Notes

- **Tuning is Paramount:** The specific weights, thresholds, probability values, and even the definitions of piece categories are highly sensitive parameters. They *must* be tuned through rigorous A/B testing³, analysis of player data (retention rates, session length, failure points), and qualitative playtesting.²¹ Start with very subtle adjustments in Tier 1 and conservative triggers for Tier 2, then iterate based on observed player behavior.
- **Computational Performance:** Analyzing the board state and calculating available placements for multiple pieces can be computationally expensive, especially on mobile devices. These algorithms need to be optimized for performance to avoid impacting the game's responsiveness. Techniques like caching board state analyses or using heuristics for placement checks might be necessary.
- **Transparency vs. Immersion:** Decide on the level of transparency for the DDA. For Tier 1, invisibility is generally preferred.²² For Tier 2, the team might consider whether a subtle visual cue or no cue is better. Explicitly telling the player the game is getting easier can break immersion and reduce satisfaction.¹⁹

7. Conclusion: Tuning for Engagement and Longevity

The analysis reveals a significant divergence in the approach to difficulty management between *Block Blast!* and *X Blocks*. *Block Blast!* leverages a sophisticated, multi-faceted DDA system, likely incorporating complex algorithms and AI³, to proactively manage the player experience. Its system appears tiered, offering continuous subtle assistance ("helpful pieces"⁷) to maintain flow and a more overt safety net ("God Mode"¹¹) to prevent excessive frustration and churn. This aligns with its stated philosophy of being a long-term, low-stress "companion" game.³

In contrast, *X Blocks* currently appears to lack an effective DDA mechanism capable of preventing the frequent and frustrating grid-lock scenarios reported by users.⁴ The perception of unfair or obstructive piece generation⁴ suggests that any existing difficulty management is either poorly tuned or non-existent, leading to abrupt,

unavoidable failures that negatively impact player satisfaction and retention.

Based on this analysis, the following actionable recommendations are proposed for the *X Blocks* team:

1. **Implement Adaptive Piece Generation:** Transition from the current piece generation method to an adaptive algorithm. This system should form the core of the DDA, dynamically adjusting piece probabilities based on real-time analysis of player state and board complexity.
2. **Prioritize Grid Lock Prevention (Tier 1):** The primary function of the adaptive algorithm should be to subtly guide players away from unwinnable grid-lock states. This involves tracking metrics like board density, available placements for upcoming pieces, and time since the last clear, and using this data to slightly increase the likelihood of receiving placeable and helpful pieces when the player shows signs of struggle.
3. **Consider a Safety Net (Tier 2):** Implement a secondary, less frequent intervention mechanism triggered by critical states (e.g., imminent grid lock predicted by lookahead, or repeated quick failures). This could involve guaranteeing a set of highly useful pieces, acting as a crucial defense against player churn caused by extreme frustration. Careful tuning is essential to ensure this doesn't overly diminish the sense of challenge.
4. **Focus on Perceived Fairness:** Design the DDA interventions to feel supportive or neutrally random, actively countering the current player perception of the game being obstructive. Avoid overly obvious manipulations and ensure the system primarily helps players overcome difficult situations rather than simply making the game trivially easy.
5. **Embrace Iteration and Data Analysis:** Recognize that implementing DDA is not a one-time task. The proposed system provides a framework, but achieving the optimal balance requires a commitment to iterative development, mirroring Hungry Studio's extensive A/B testing.³ Continuously monitor player metrics (session length, retention, failure rates at different *Struggle_Score* levels), gather player feedback, and use this data to relentlessly tune the DDA parameters (weights, thresholds, probabilities) until the desired player experience – reduced frustration and improved engagement – is achieved.

By adopting a player-centric DDA strategy focused on proactively managing difficulty and preventing frustrating failure states, *X Blocks* can significantly improve its player experience, moving closer to the engaging and retentive qualities observed in successful casual puzzle games like *Block Blast!*. The goal is not merely to replicate another game's system, but to leverage DDA principles to build a uniquely satisfying

and enduring experience for *X Blocks*' own player base.

Works cited

1. Dynamic game difficulty balancing - Wikipedia, accessed April 29, 2025, https://en.wikipedia.org/wiki/Dynamic_game_difficulty_balancing
2. (PDF) Exploring Dynamic Difficulty Adjustment in Videogames - ResearchGate, accessed April 29, 2025, https://www.researchgate.net/publication/339174028_Exploring_Dynamic_Difficulty_Adjustment_in_Videogames
3. How Block Blast! Shattered Records with AI, A/B Testing - and a ..., accessed April 29, 2025, <https://www.globalgamesforum.com/features/how-block-blast-shattered-record-s-with-ai-a-b-testing-and-a-bold-bet-on-long-term-players>
4. X Block - Brain Puzzle Game - Apps on Google Play, accessed April 29, 2025, <https://play.google.com/store/apps/details?id=net.ibexsolutions.venus>
5. X Block - Brain Puzzle Game - Apps on Google Play, accessed April 29, 2025, <https://play.google.com/store/apps/details?id=net.ibexsolutions.venus&hl=en>
6. Dynamic Difficulty Adjustment for Maximized Engagement in Digital Games - ResearchGate, accessed April 29, 2025, https://www.researchgate.net/publication/322414816_Dynamic_Difficulty_Adjustment_for_Maximized_Engagement_in_Digital_Games
7. Block Blast by Hungry Studio is doing \$1M a Day - Gamigion, accessed April 29, 2025, <https://www.gamigion.com/block-blast-by-hungry-studio-is-doing-1m-a-day/>
8. Block Blast! - Apps on Google Play, accessed April 29, 2025, <https://play.google.com/store/apps/details?id=com.block.juggle>
9. Block Blast! - Apps on Google Play, accessed April 29, 2025, <https://play.google.com/store/apps/details?id=com.block.juggle&hl=en>
10. How Block Blast! Shattered Records with AI, A/B Testing - and a Bold Bet on Long-Term Players | Gamesforum, accessed April 29, 2025, <https://www.globalgamesforum.com/news/how-block-blast-shattered-records-with-ai-a-b-testing-and-a-bold-bet-on-long-term-players>
11. How could Block Blast by Hungry Studio earn more? Monetization ..., accessed April 29, 2025, <https://balancy.co/blog/2025/03/26/how-could-block-blast-by-hungry-studio-earn-more-monetization-and-gameplay-deconstruction/>
12. Build a Puzzle Game Like Block Blast: Features, Process & Cost! - Apptunix, accessed April 29, 2025, <https://www.apptunix.com/blog/develop-a-game-like-block-blast/>
13. Block Blast-Block Puzzle Games Reviews - Metacritic, accessed April 29, 2025, <https://www.metacritic.com/game/block-blast-block-puzzle-games/>
14. Dive Into Block Blast! User Feedback Analysis Report - Kimola, accessed April 29, 2025, <https://kimola.com/reports/dive-into-block-blast-user-feedback-analysis-report->

- [google-play-152706](#)
15. Color Block Blast - APK Download for Android | Aptoide, accessed April 29, 2025, <https://89-blocks.en.aptoide.com/app>
 16. X Block - Brain Puzzle Game | Télécharger et jouer sur PC - Google Play Store, accessed April 29, 2025, <https://play.google.com/pc-store/games/details?id=net.ibexsolutions.venus&hl=fr>
 17. Dynamic Difficulty Adjustment - Game Design and Theory - GameDev.net, accessed April 29, 2025, <https://gamedev.net/forums/topic/666665-dynamic-difficulty-adjustment/>
 18. Dynamic Game Difficulty Adjustment. : r/truегaming - Reddit, accessed April 29, 2025, https://www.reddit.com/r/truегaming/comments/oz14q/dynamic_game_difficulty_adjustment/
 19. Dynamic Difficulty Adjustment: The Participation Trophy of Game Design - Wayline, accessed April 29, 2025, <https://www.wayline.io/blog/dynamic-difficulty-adjustment-participation-trophy>
 20. user.ceng.metu.edu.tr, accessed April 29, 2025, https://user.ceng.metu.edu.tr/~tcan/se542_f1718/Schedule/reading4.pdf
 21. External Assistance Techniques That Target Core Game Tasks for Balancing Game Difficulty - Frontiers, accessed April 29, 2025, <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2020.00017/full>
 22. Dynamic Difficulty Adjustment: The Future of Personalized Gaming - Wayline, accessed April 29, 2025, <https://www.wayline.io/blog/dynamic-difficulty-adjustment-personalized-gaming>
 23. Top 13 Block Puzzle Strategy and Tips to Attain High Scores - MPL Games, accessed April 29, 2025, <https://www.mplgames.com/blog/block-puzzle-strategy/>
 24. How to Play Block Puzzle Game: Gameplay and Rules, accessed April 29, 2025, <https://www.mplgames.com/blog/how-to-play-block-puzzle-game/>
 25. Hooked on "Block Blast" on mobile right now. Questions regarding design? - Reddit, accessed April 29, 2025, https://www.reddit.com/r/gamedesign/comments/1b2ki93/hooked_on_block_blast_on_mobile_right_now/
 26. How to Get a High Score in the Block Puzzle Game Online? - MPL, accessed April 29, 2025, <https://www.mpl.live/blog/get-high-score-in-block-puzzle-game-online/>
 27. What is the block algorithm for Classic Tetris? - Reddit, accessed April 29, 2025, https://www.reddit.com/r/Tetris/comments/6o6tvv/what_is_the_block_algorithm_f_or_classic_tetris/